

Graduation Report: Software Architecture Energy Configurator



Student: Wiebe Sportel
Study Number: 513001
Graduation Coach: Yvens Rebouças Serpa
Company Coach: Wander Lamot
Company Name: Aeres Tech Productions

Abstract

Aeres Tech Productions produces educational modules for agriculture. There is a need to go beyond e-learning modules towards a tool that students can use to simulate a farm in real life. The **Energy Configurator** aims to create an interactive application where the student can set up a farm and see its financial impacts.

This report covers the process of creating this application from scratch. It discusses what architectural decisions were made during this project, such as maintaining a clear and modular code structure, and how these were addressed using software architecture and good coding practices—for example, implementing drag-and-drop features. The application was tested, and it produced promising results, meeting the necessary software requirements. The report concludes that a strong focus on software architecture can enhance many parts of the codebase. Benefits include a faster development cycle, abstracted components for easier expansion, and smoother integration of new features.

Contents

1. Introduction	5
1.1 Company.....	5
1.2 Problem Analysis.....	6
1.3 Research Question	6
1.4 Scope	6
2. Theory	7
2.1 Software Architecture.....	7
2.1.1 Role of Software Architecture	7
2.2 Software Sustainability	7
2.3 Code Quality	8
2.3.1 Measuring code quality	8
2.4 Game Architecture	8
2.4.1 Bad architecture and its consequences.	8
2.4.2 Preventing bad architecture.....	8
3. Development	9
3.1 General Applications	9
3.1.1 Sequence System.....	9
3.1.2 Worker System	9
3.1.3 Usability System	11
3.1.4 Transmission System.....	11
3.1.5 Advanced Debugger	11
3.1.6 Saving System.....	12
3.1.7 Calculate System.....	12
3.2 The Sequence System.....	12
3.2.1 The Settings	13
3.2.2 The steps	13
3.2.3 The Conditions and Rules.....	13
3.3 The Worker System	14
3.3.1 The Search.....	15
3.3.2 Execution.....	15
3.4 The Transmission System.....	15
3.4.1 The Transmission Manager	15
3.4.2 The Transmission Invoker.....	15
3.4.3 The Transmission Connector	16
3.5 The Saving System	17

3.6 The Calculate System.....	17
4. Testing	18
4.1 Technical tests preface.....	18
4.2 Testing parameters	18
4.3 Technical tests	19
4.3.1 The Worker System	19
4.3.2 The Sequence System	20
4.3.3 The Transmission System	20
4.3.4 Technical Test Retrospective	21
4.4 User Tests.....	21
4.4.1 The mockup.....	21
4.4.2 The User Tests	25
4.4.3 User Tests retrospective.....	26
5. Reflection, Conclusion & Future Work	27
5.1 Conclusion	27
5.1.1 Main Research Question	27
5.1.2 How to make a modular codebase?	27
5.1.3 How to save and load inside of the application in a dynamic and adaptable way?	27
5.1.4 How to export the bill into PDF form, both for windows and web?.....	27
5.2 Future work.....	28
5.3 Reflection	28
Resources	29

1. Introduction

Aeres Tech Productions (*Over Ons*, z.d.) is a company that operates within the dynamic landscape of educational solutions. Aeres Tech Productions cater mainly towards vocational training institutions and business to business clientele. They stand at the forefront of innovative learning methodologies and specializing in niche sectors such as refrigeration technology, agricultural machinery and the burgeoning green sector, which positions them in a unique spot to meet the needs of their clients. The primary focus of work lies in delivering tailored learning solutions for MBO students in the Netherlands, equipping them with the theoretical knowledge and practical skills required to excel in their respective fields.

Aeres Tech Productions produces a series of e-learning modules designed to educate both current and future farmers on all aspects of energy use and management on the farm, catered towards MBO students. Aeres Tech Productions produce educational modules for agriculture. They also have an interactive app for it ready made within Unity, it's called the **Farm Builder**. The **Farm Builder** works like a app of sorts, allowing users to design their own virtual farms and see how their decisions impact financial outcomes, sustainability and animal welfare.

Aeres Tech Productions has a need to expand this prototype, as they claim it is not detailed enough. The prototype allows you to change aspects of the farm, without seeing the consequences of doing so in further detail, like seeing the impact on an energy bill once the user introduces another device or energy saving solution onto the farm. Other requested features are, for example, print the bill to a PDF, and save/load their setups inside the prototype for later showcase/use.

Aeres Tech Productions wants to implement an **Energy Configurator** inside of this prototype. An energy configurator in this context is a concept that enables users to manage energy on a virtual farm. It visualizes the impact of energy production and consumption, which is then reflected in a simulated bill.

Moreover the prototype has some problems, the main problem is that the company can't access its source code since it contains their intellectual property (IP). With that in mind, Aeres Tech Productions has requested to put a focus on making the app expandable for future developers and providing access to the source. This introduces different kinds of challenges, mainly related to Software Architecture, Software Sustainability, Code Quality, and Game Architecture. This project is called the **Farmer Finance Project**.

1.1 Company

Through collaboration, with industry experts and leveraging the latest advancements in educational technology, Aeres Tech Productions are committed to empower the next generation of professionals in these vital industries.



Figure 1: Aeres Tech Productions company logo.

Aeres Tech Productions (Aeres Tech Productions, n.d.) specializes in crafting diverse educational solutions. Ranging from traditional books to cutting-edge e-learning modules as well as immersive virtual reality and augmented reality experiences. Aeres Tech Productions is really diverse in content creation. These products/solutions are made mainly based upon the customers wishes and the best

solution to that wish. The company has existed for over 25 years, however 4 years ago, they were taken over by Aeres Tech. Before this, the company was named **Acole** and they focussed mainly on development for Educational Institutions and Business to Business.

1.2 Problem Analysis

Aeres Tech Productions wants to expand on the **Farm Builder** prototype. After discussing the prototype in detail, there are pain points of Aeres Tech productions that should be considered: it is not possible to save/load progress in the prototype, leading to frustration as users are not able to work from where they left off; it is possible to see the financial impact of the farm, but not in enough detail such as having it in an energy bill, like in real life; it is not possible to expand the prototype, due to the initial producer of the app not wanting to share the source because it contains their IP; it is not possible to run this app in web, this is troublesome due to their e-learning modules being present on the web. They would rather have it all in one place, however this point might be out of scope for the assignment depending on a few unknown factors.

The need for a modular codebase that is dynamic is because of the fact that I might not work there in the future, so having it set up in this manner will allow for future developers to easily work with the tools provided and easily make changes and additions.

The importance of this assignment is that at the end of it, a product is delivered that they can use to educate students about finance on a farm, see the consequences of actions performed on that farm, be able to save/load their progress to work on later. Alongside a codebase that can be expanded upon later down the line.

1.3 Research Question

How can software architecture be used to design and implement the Farm Builder system to guarantee a modular codebase that is dynamic and easily adapts to new features being introduced allowing for users to save/load their progress and view the financial side of the farm in PDF form, and help future developers maintain and expand upon the codebase in an easy manner?

To answer the research the following sub questions were created:

1. How to make a modular codebase?
2. How to save and load inside of the application in a dynamic and adaptable way?
3. How to export the bill into PDF form, both for Windows and Web?

1.4 Scope

The scope of this project is as follows: Creating the application with the energy configurator with save/load functionalities; Implementing web support, creating PDF web support is out of scope due to the amount of research being required to implement the necessary Javascript libraries to allow the creation of a PDF; Testing with the target audience is outside of the scope due to the need for the real content of the application, but the application was tested internally with users to guarantee its functionality and goals; Lastly, expanding upon existing systems made in the previous internship.

All the graphics for the project were provided by the company's artist which is not a game artist. All the effort related to the graphics were done simply to have them available as requested by the company without significant effort in optimizing them for the system nor on workflows to add them to the system.

2. Theory

2.1 Software Architecture

Garlan, D. (2008) found that, the term ‘software architecture’ is used in different ways, often making it difficult to understand what aspect of the ‘software’ is addressed. Among these uses are the architecture of particular systems; an architectural style. (Think client-server structure for example); the general study of architecture.

Within software engineering, most uses of the term focus on these interpretations. A typical definition is:

The software architecture of a program or computing system is the structure or structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them.

While there are of course similar definitions of software architecture, at the core of it all is the notion that the architecture of a system describes its gross structure using one or more views supported by Garlan, D. (2008).

2.1.1 Role of Software Architecture

Software architecture plays a key role as a bridge between requirements and code. Giving an abstract description (or model) of a system that exposes certain properties, while at the same time hiding others. This should make for a guide to the overall system, and allows designers to reason about the ability of a system to satisfy certain requirements, and suggest a blueprint for system construction and composition as supported by Garlan, D. (2008).

It can also play an important role in at least 6 aspects, relevant towards software development:

Understanding, the architecture should simplify our ability to understand large systems by presenting them in such a way that a systems design can be easily understood; **Reuse**, component libraries, alongside with frameworks into which components can be integrated; **Construction**, an architectural description provides a partial blueprint of major software components and dependencies between them; **Evolution**, easily change connection mechanisms to handle evolving concerns about performance and reuse; **Analysis**, architectural descriptions provide new opportunities for analysis; **Management**, critical evaluation of an architecture most of the time typically leads to a much clearer understanding of requirements, implementation, strategies, and potential risk; **Communication**, an architectural description often serves as a vehicle among stakeholders.

2.2 Software Sustainability

Software Sustainability, refers to the capacity of a system to endure. Longevity, an expression of time and the ability to maintain are key factors at the heart of sustainability. In relation to software, there are at least two distinct viewpoints for the topic of software sustainability. These are as follows:

In general, however, the concept of sustainability is not widely understood, in some cases even misunderstood, in the field of software engineering with opposed views on its meaning in the software engineering community. It can be focussed down further into these three concepts:

Software Sustainability; the capacity of the software-intensive system itself to endure will be a concern for the operating organisation and community; **Software Architecture Sustainability**, the capacity of that software system to endure in turn is contingent upon its structures and their ability to evolve; **Sustainable Software Architecture Design Decisions**, this architecture, in turn, reflects the

foundational design decisions that structure the system and its elements, so decision making is increasingly a focus of attention.

2.3 Code Quality

Code quality is a way to talk about how efficient, readable, and usable code is. Code quality measures the accuracy and reliability of code. Code quality is important due to a number of factors, it represents the efficiency of code, not just in its functionality but also readability and long-term management (Amazon Web Services. n.d.)

2.3.1 Measuring code quality

By combining quantitative and qualitative assessments, it's possible to attain the best measure of software quality (Amazon Web Services. n.d.). There are five key code quality areas of measurement: **Reliable**, code runs as documented each and every time you run it. It handles unexpected inputs and interruptions without crashing or other rogue behaviours; **Extendible**, it's easy for the code to be updated or modified after it's complete; **Testable**, a piece of code should be easy to develop tests for and run tests on; **Portable**, making the code be able to work in another environment; **Reusable**, High quality code is modular and designed for reuse. (Amazon Web Services. n.d.).

2.4 Game Architecture

Video games are composed of many different components, these components are, and not limited to; graphics, audio, animation, environment and scripting. These are computer programs provided by game engines and exposed as APIs (Application Programming Interface). Game developers work with those APIs and combine their functionalities, to create video games as said by Pham, D. V. (2020).

By combining them, we form some kind of structure or system. These programs, the relationships between them and the human discipline to follow as well as maintain the design are defined as software architecture explains Pham, D. V. (2020). Since video games are more or less software, we have the term game architecture to express the intent of software architecture in the context of video game development explains Pham, D. V. (2020).

2.4.1 Bad architecture and its consequences.

Usually, the codebase of a project with a bad architecture falls into one of these two categories: **Spaghetti Codebase**, Code is written in an unstructured and spontaneous manner. Parts of the game reference each other without clear intention being present. They gradually form into a woven thread of codes. It makes the codebase extremely, even in some cases impossible to understand; **Rigid Codebase**, unlike the spaghetti codebase, this one usually has a structure. The biggest problem is that this system does not embrace nor anticipates changes. Yet these changes are hard to avoid in an ever evolving development space.

Bad architectures are always painful to deal with, that does not mean however that projects with good architecture design are always great to work with. It requires a lot of effort to design, alongside that it requires discipline to maintain its elegant, well-organized structure.

2.4.2 Preventing bad architecture

Good architectures advocate modularity, to solve the problem that leads you to bad architecture. This specifically entails the way functionalities rely on each other, one thing changing could affect anything it depends on. This comes at a cost, it adds another layer of abstraction and indirection that can put a tax on performance and obscurity on the code logic according to Pham, D. V. (2020).

3. Development

3.1 General Applications

All of the basic functionalities and the general use case for the systems that have been developed will be described here.

3.1.1 Sequence System

The sequence system is used in all parts of the application where it is applicable, if there is a system that could follow a sequence of steps, it is used there. An example of this would be the modal sequence, where if you click on an eye in the application, a modal opens where it instantiates a prefab with the responsible content inside of it, and removes it afterwards when closed.

The sequence system also allows for dynamic changes or edits, the sidebar in the application has 2 sequences coupled to it, 1 sequence for opening and displaying the content inside, and a sequence for dynamically changing what content is displayed while open.

The sequence system has an abstract definition, to make sure it is not too rigid or steadfast in its usage across the app to prevent similar systems from being made that serve the same purpose. This system was developed in my previous internship and makes a comeback in this project, with additional features that will be discussed later, as seen in Figure 2.

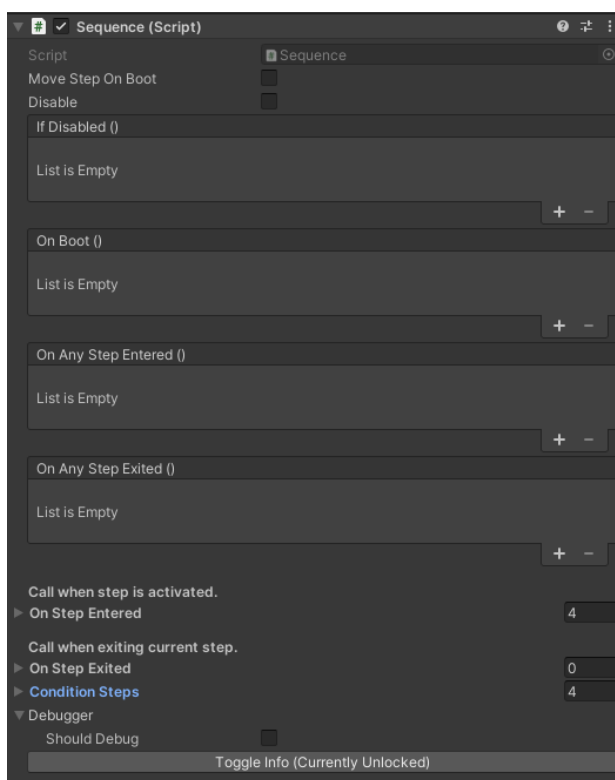


Figure 2: Sequence System in the editor.

3.1.2 Worker System

This system is used in all parts of the application, most of which is applicable. The worker system allows you to perform a method upon all objects of a specific type, with conditions being involved. An example of this would be with opening the modals, modals are panels containing information about a

specific device on the farm. When a modal is opened, the modal opens from the eye, where it was clicked. However, if another eye was clicked on while it was opening, the animation would snap towards that position and continue from there. The worker system allows you to counter this by for example, telling every single **Point of Interest** (POI) to not be interactable when one is clicked. A **Point of Interest** (POI) is a green or orange eye. Green standing for energy production, and orange standing for energy consumption. It is responsible for opening the modal, shown in Figure 3.



Figure 3: Point of Interest, green and orange eyes. Green is energy production, orange is energy consumption.

The worker system allows for more complex operations inside the app as well, such as the modal content, the modal content is dynamically instantiated with a prefab through a callback that is coupled on each POI.

The worker system has an abstract definition and can be used anywhere in the application to perform complex operations, or quickly fix problems before they even start occurring. This also allows for small code samples to that serve one purpose to be non-existent, as small mechanics can be handled by the worker system, an example of such an operation is shown in Figure 4.

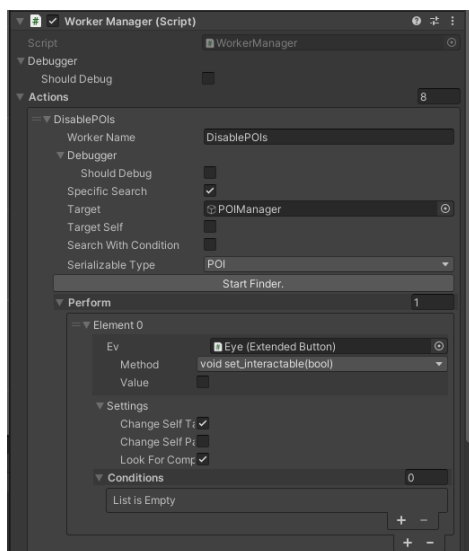


Figure 4: Worker operation to disable all POIs

3.1.3 Usability System

Usables are objects that represent data and are used to store data in the application. Usables are used in all kinds of manners to dynamically change and update different aspects of the app, such as the amount of windmills on the screen, the amount of windmills in the device legend, the count of it in the modal, etc.

The Usables can be kind of viewed as helpers for different parts of the application to function in a dynamic manner. Usables are not scriptable objects, as they do not meet the criteria to do so, each usable is different and has different parameters. Making the Usable a struct would also make no sense as much of its data cannot be immutable. They are MonoBehaviours. They were implemented as MonoBehaviours due to concerns that it would conflict where it wouldn't work on scriptable objects. This is because of a library I am using to add serializable callbacks.

Usables can be described as variables inside of the editor as said before they are not structs or scriptable objects as they don't meet that criteria. These Usables are used all across the application, but are still very centralized in their structure, making them easy to find in the application such as seen in Figure 5.

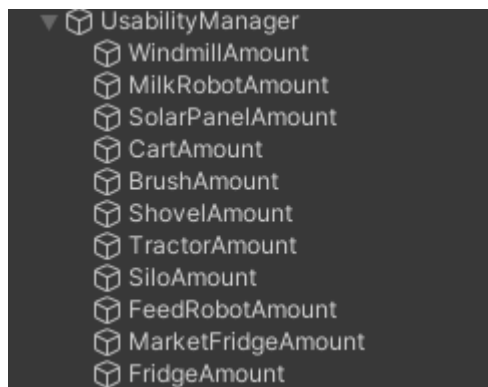


Figure 5: Usability Structure.

3.1.4 Transmission System

The transmission system is designed to communicate with prefabs that have just instantiated and communicate with the scene, or vice versa. It is used for many things inside of the application, most of them, prefab related.

As an example, the modal content prefab contains a plus (+) button, which does nothing by itself. But by using the transmission system, it allows you to connect that plus (+) button to an action in the scene, such as increasing the amount of windmills present. Or for example telling the modal content prefab that the slider it contains needs to have a max value when instantiated.

3.1.5 Advanced Debugger

The advanced debugger is a debugger meant to replace the Unity **Debug.Log**. It functions as a more personalized debugger to the object it is present on. The advanced debugger prints the script and the game object that it is coupled to whenever you log a message, solving the issue of not being able to identify which object is logging which information. That is especially helpful in scenarios in which multiple objects are constantly printing logs on the console.

For code quality to rise, a piece of code should be testable based upon its execution, however what is also important is tracking any kind of potential issues that may arise. Logging itself can get very

confusing very quickly, this is what I have experienced in other projects and this one itself, I just wanted to debug something quickly and remove it later, then I would forget and the logs would just stack up. But now, there is a clear distinction, shown in Figure 6.

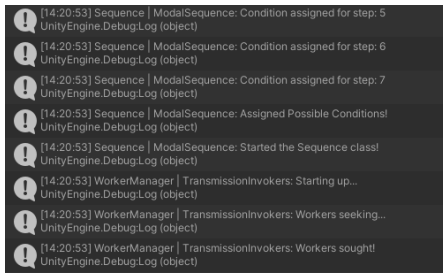


Figure 6: Advanced Debugger prints.

Every single component I made has this debugger, you can turn it on or off per script you wish to debug. If you have 1 specific sequence you have issues with, you can turn that on for debugging without affecting the rest of the sequences. The debugger displays it's message differently from regular debugging. In the log you would have to click a message to know where a debug was coming from, now though you can directly see it. You first see the class it came from, and then what game object the code is present on. It allows the

developer to quickly discern where potential issues may arise and quickly debug through a scripts functionalities.

The advanced debugger is a serializable class that you place upon the script that you want to debug. And in the **Start** method you pass the object name and class name.

3.1.6 Saving System

The saving system is designed to solve the issue of saving data of the current app state in a dynamic manner. The system works in collaboration as to how the usability system is structured to save values into a JSON file.

As an example, let us assume a scenario in which there are 10 usable variables, these usables have built-in variables that can be returned. When assigned to the saving system, it allows saving those 10 usables in a JSON file, but also load them back in and determine where they need to go in the scene.

3.1.7 Calculate System

The calculate system serves the purpose of creating a calculation in the editor. It allows the developer to make complex calculations and have it dynamically update a part of the application. It solves the issue of displaying a proper bill, the bill will contain many kinds of calculations and doing this through code is a chore, due to it taking a lot of time.

As an example, let us assume that we need to calculate how much money we would get back from windmills based on the farm, 2 values can be set while 1 of them is dynamic, namely the amount of windmills. When you make the calculation and call it, It will create that calculation and give you a result based upon those 2 values and the number of windmills.

3.2 The Sequence System

This system makes sure that you can easily create systems and a proceed action with the tools that Unity already provides, without you needing to program much. Here are 2 examples of how the sequence system is used, and what for:

1. The modal system, the modals can be opened and closed with animations.
2. The energy configurator, the sequence is responsible for displaying responses on the template if device amount has changed.

3.2.1 The Settings

There are 4 events that give control on what generally happens in the Sequence: the **IfDisabled** event performs when the sequence is disabled, this not only allowing to quickly debug parts of the application, but also turn them off if they are interfering with something else; The **OnBoot** performs 1 frame after the start method is done, allowing for additional setup to take place without the sequence system breaking in the process because it's not fully set up yet; **OnAnyStepEntered**, allows you to bind an action when the sequence enters a new step, and; **OnAnyStepExited**, allows you to bind an action when the sequence exits a step.

3.2.2 The steps

Every step has this structure, as seen in Figure 7 and 8.

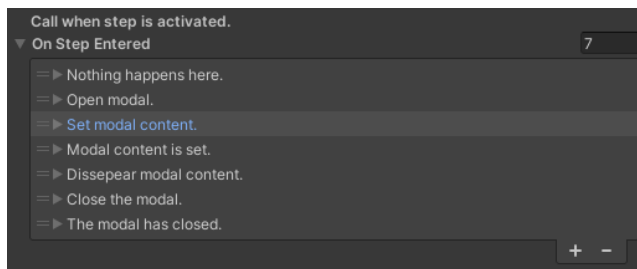


Figure 7: Steps in list form.

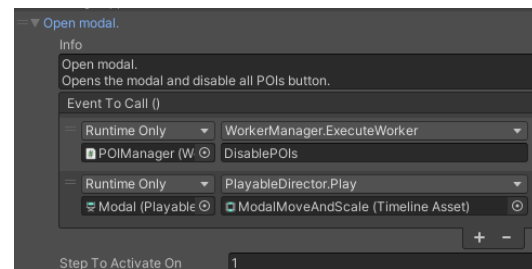


Figure 8: Individual Step

The step has some info, containing the details of the step and what purpose it has. This allows the developer to give your own definition and explanation for the individual step, allowing anyone to take one look at the step, and know what it's responsible for.

3.2.3 The Conditions and Rules

The way a sequence progresses is through steps by using conditions.

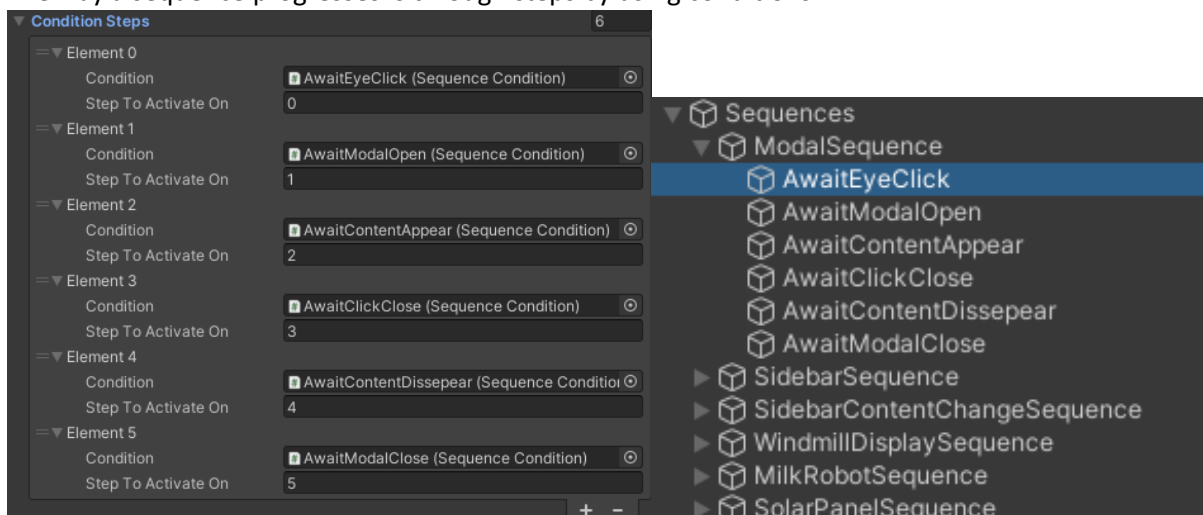
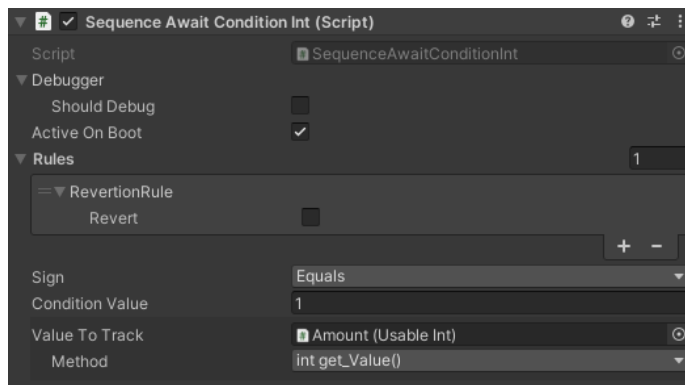


Figure 9: Step Conditions in list form.

Each step can have multiple conditions it needs to meet. For example, step 0 can have 3 conditions, while step 1 only has 1, as seen in Figure 9. The conditions of a sequence, for clarity, should be structured something along the lines of this in the editor, as seen in Figure 9.

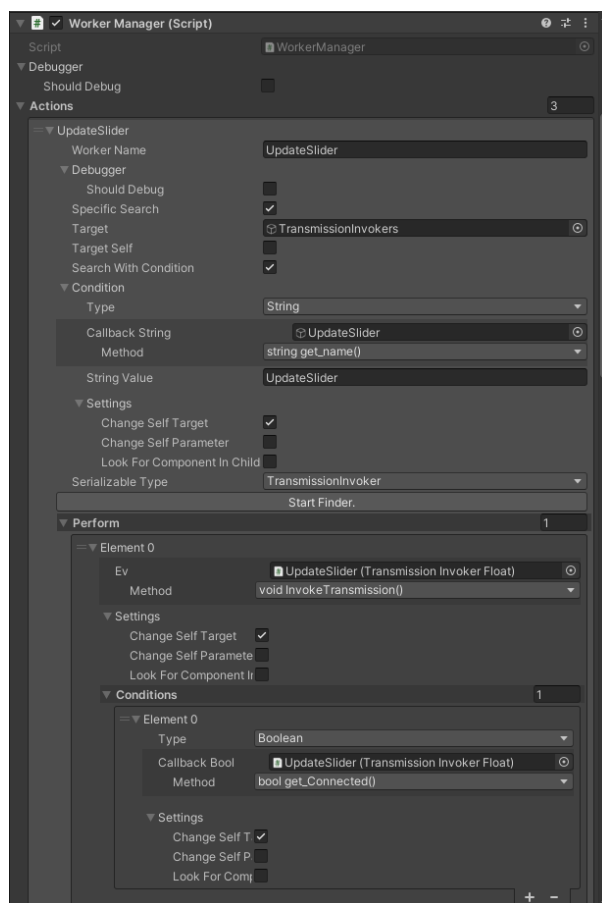
This structure allows you to quickly find where all the conditions for that specific sequence are located. There are different kinds of conditions, these are as follows: **SequenceCondition**, a condition that awaits an invocation from another part of the application; **SequenceAwaitConditionBool**, a condition that awaits until a callback value becomes true; **SequenceAwaitConditionInt**, a condition that awaits for a int to meet the conditions described in the editor. A sequence condition can have rules. These rules indicate whether the condition should adhere different behaviour then it normally should. These rules are currently in place: the multiple completions rule, whether a condition should meet its criteria multiple times, and how many times before it moves onto the next step; the reversion rule, should a step be revertible, such as going back a step depending on a condition.



This condition awaits until the specific value described in the callback becomes equal to 1. When it does, the condition is called, as shown in Figure 10.

Figure 10: Sequence Condition Await Int and Rules

3.3 The Worker System



The worker system allows you to search for objects of a specific type, and execute an operation upon all of the objects found, shown in Figure 11.

At the start of the app, the worker searches for objects specified by a **SerializableType** to look for, with a certain condition if so needed.

A **Worker** starts with a name, you can execute a worker through the editor based upon that name.

Once done so, a worker will then go through all the objects it has found at the start of the app, and it attempts to invoke the assigned method, in Figure 11 this is **InvokeTransmission()** method.

Figure 11: The Worker Manager.

3.3.1 The Search

Searching what objects the **Worker** needs is done at the start of the app which is mainly done through inputting a **SerializableType**, this can has a multitude of options on how to look for these objects: the system can perform a global search across the entire scene; you enable the specific search option, to choose a target and look through all of its children, it can be itself or an outside target; you can seek an object based upon a specific condition, for example, if there are a multitude of objects named UpdateSlider, and you want to look for those with that name, you can.

3.3.2 Execution

The execution for the **Worker** has some settings you can use to your advantage, the **change self-target** option changes the target of the method to the next object. For example, search for the POI object, and you find 2. Promptly named poi1 and poi2. When enabled, it will execute the method assigned in performs on both poi1 and poi2. If this was not the case it would just execute the method on poi1, twice.

When **change self-parameter** is enabled, if the method has a parameter, it will change that parameter to the found objects from the beginning. Say I have a POIManager, this POI manager has a method with an argument, namely a POI object. That methods argument will be replaced with poi1 and poi2.

Look for component in children is pretty straight forward, if the target type does not have the script on itself I am trying to execute the method on, look for it in its children.

You can also have a multitude of conditions that the worker needs to meet before it can execute the method. The conditions work in the same manner as described up above, with the same settings.

3.4 The Transmission System

3.4.1 The Transmission Manager

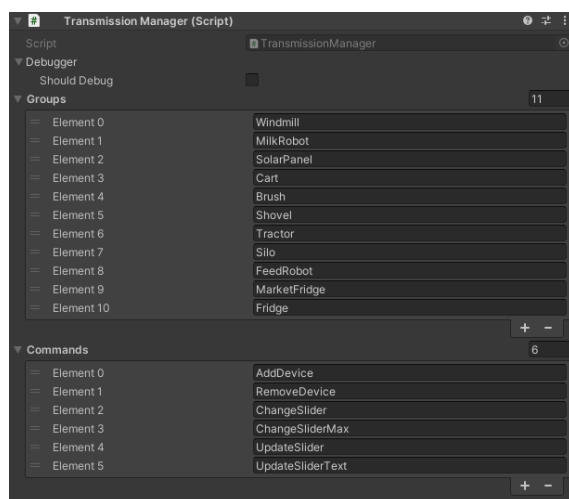


Figure 12: The Transmission Manager.

As seen in Figure 12, the transmission manager has 3 responsibilities, the definition of groups, the definition of commands, and the responsibility of making the connection from the instantiated prefab to the scene.

The transmission manager at the start records all connectors and invokers present in the scene. When another connector or invoker is instantiated, the transmission manager will attempt to connect them to their respective counterpart, a connector or invoker with the same group and command.

3.4.2 The Transmission Invoker

The transmission invoker is a script that is responsible for communicating where the original invocation comes from, imagine it like this, if we have a prefab, that is the modal content for the

windmill, a plus (+) and minus (–) button to increase or decrease the amount for the windmill, inside the prefab it can't do anything in the scene, but the transmission invoke change that as shown in Figure 13.

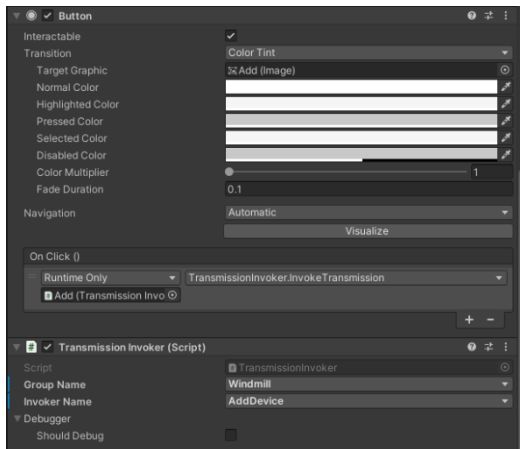


Figure 13: The Transmission Invoker.

The transmission invoker is activated with a button click and calls for the event that is determined by its properties. This is then communicated to a transmission connector that something has occurred, which I will elaborate on later.

There are also different kinds of invokers such as seen in Figure 13. This particular invoker inherits from the transmission invoker generic variant class, allowing for there to be multiple different implementations, that allows for additional invocations with parameters. This one, being invoked with a float.

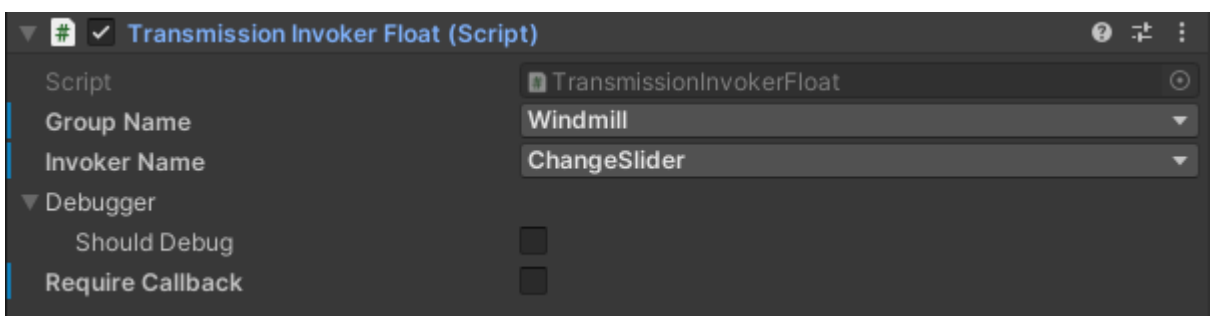


Figure 14: Transmission Invoker Generic.

3.4.3 The Transmission Connector

The transmission connector is a script that is responsible for communicating which invoker it is responsible to. The transmission connector is where the actual communication part of the destination happens, the invoker is the communicator, telling the connector to do something, whether it be inside a scene, or inside a prefab. In Figure 13 and 14, you have seen the transmission invokers, below is their connector counterpart, shown in Figure 15 and 16.

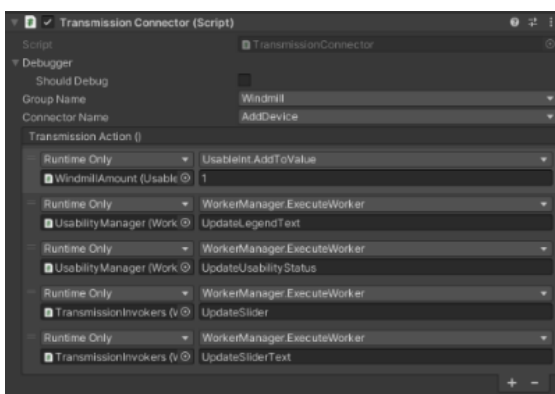


Figure 15: Transmission Connector.

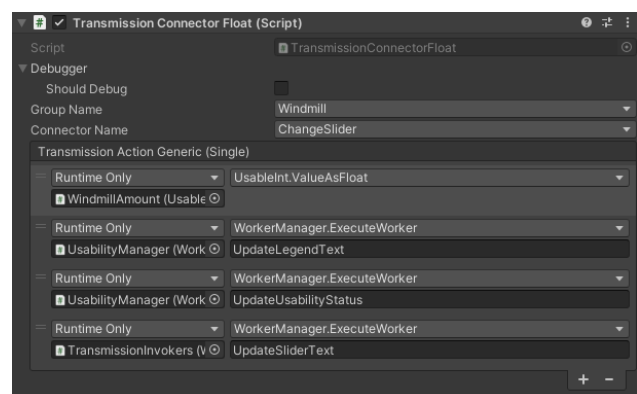


Figure 16: Transmission Connector.

3.5 The Saving System

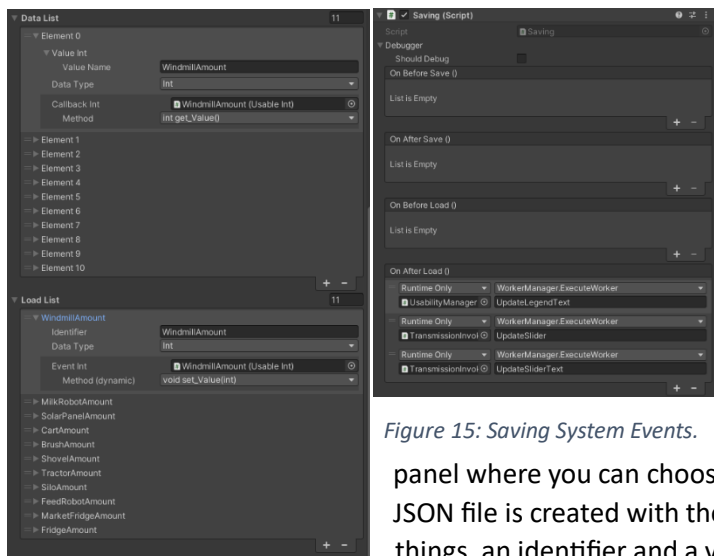


Figure 14: Saving and Loading.

The saving system has a few events to start on, which are **OnBeforeSave**, **OnAfterSave**, **OnBeforeLoad**, **OnAfterLoad** all of these events are self-explanatory. These events are displayed in Figure 17.

The data list has entries, entries that should be saved inside of a JSON file when asked to. When the **Save()** function is called, it will save all of these entries as JSON and open a file

Figure 15: Saving System Events.

panel where you can choose your save destination. Once done so, a JSON file is created with the relevant data inside of it. The data has 2 things, an identifier and a value of a specific type, shown in Figure 18.

Once the load function is called, it will load all the variables back from the chosen JSON file. Once done so, it will check if the save value has the same identifier and type, then perform whatever the event is defined to do. In this case, it's setting the windmill amount, shown in Figure 15.

3.6 The Calculate System

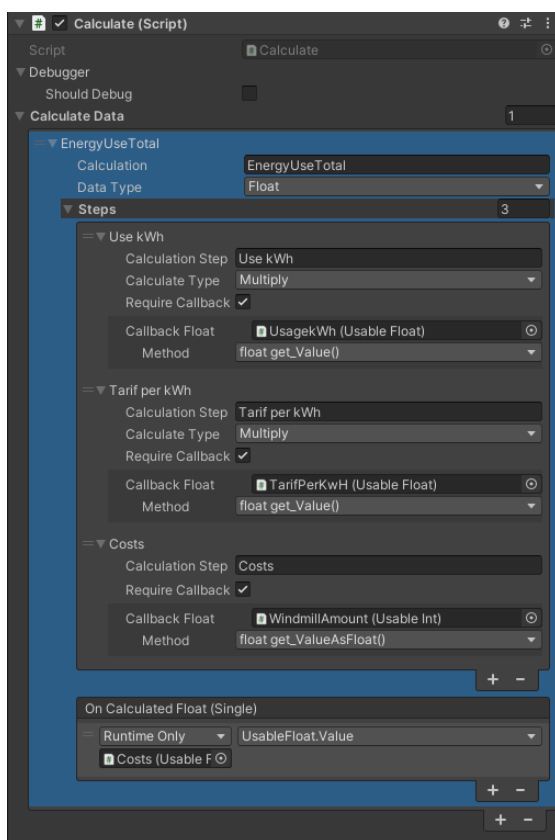


Figure 16: The Calculate System and Steps.

The calculate system has a list of entries called **CalculateData**, the calculate data contains an identifier by which the calculation is performed and a type, which represents what the type of calculation is, this can be int, float or double. Otherwise the implementation will not display the correct values. Based upon what type is selected, it will also invoke an event with the returning number made from the calculation.

The calculate system calculates certain values with each other based on a few parameters: the calculate step, which describes what the value represents; the calculate type, which is the operation to perform, such as plus, minus, division and multiplication to perform with the next number present in the list; and whether it requires a callback or not. If a callback is not required, a field will be displayed where you can manually fill in a number.

A list of these steps and a complete calculation can be viewed in Figure 19.

4. Testing

4.1 Technical tests preface

The sequence system relies on reflection to perform its operations, specifically the events stored. These are not performed on mass scale, but it is important to check what effect they have on the applications performance. This especially applies to the value conditions, these actively use reflection.

There are multiple ways to test an application, for example, the most basic way of testing, is creating a timestamp before a method is executed, and after a method is executed to check the amount of time that has passed. This does not say much, however, as it takes a lot of tests to remove any kind of variables that might be interfering with the results and get a proper **median**. Especially in Unity's case, where there are many background processes.

The solution that was chosen was the **Performance Testing Extension** (Unity Technologies. n.d.-a). This extension builds upon the testing framework for Unity, by adding onto the testing functionality it provides and allows a developer to track the performance of many aspects of a game/app. Such as the following: testing the scope of some code; testing a specific range or a single method; testing the entire performance of a scene. These are just some examples but the functionality of a single method is perfect for this use case.

What the tests will consist of is how long it takes to perform an operation, this will indicate its performance, as in how fast it is in executing that specific task. The following systems will have their performance tracked and checked, as these actively use reflection: The Worker System; The Sequence System; The Transmission System.

4.2 Testing parameters

Before the tests are performed, an explanation of the used parameters need to be given before the tests take place, so that proper understanding of the system is given. The parameters are as follows: **WarmupCount**, number of times to execute before measurements are collected. If unspecified, a default warmup is executed for 100 milliseconds or until at least 3 method executions have completed, whichever is longer; **MeasurementCount**, number of measurements to capture, defaults to 9 if not specified; **IterationsPerMeasurement**, number of method executions per measurement to use. If this value is not specified, the method is executed as many times as possible until approximately 100 milliseconds has elapsed (Unity Technologies. n.d.-b).

The parameters I used for this test were as follows, the warmup count was 5, the measurement count was 20, and the iterations per measurement was set at 1000. This meant that in total, all of the operations will be done 20000 times in total.

It is expected that the tests will be unstable (**Standard Deviation** above 5%), as we are dealing with reflections at run time, these calls are more performance intensive then their normal counterpart.

For further context, most games can run at 60 frames per second. Each frame then takes around 16.7 milliseconds to render. This is to compare the systems performance to a game with 60 fps and how this system would effect it.

4.3 Technical tests

4.3.1 The Worker System

4.3.1.1 Disable Pois (Without conditions)

The following graph (Figure 17 and 18) shows the operation of **ExecuteWorker("DisablePois")** it's job is to turn every point of interest in the scene's intractability to false. Figure 17 shows the disabling of one poi, and Figure 18 shows the disabling of 11 POI's at once.

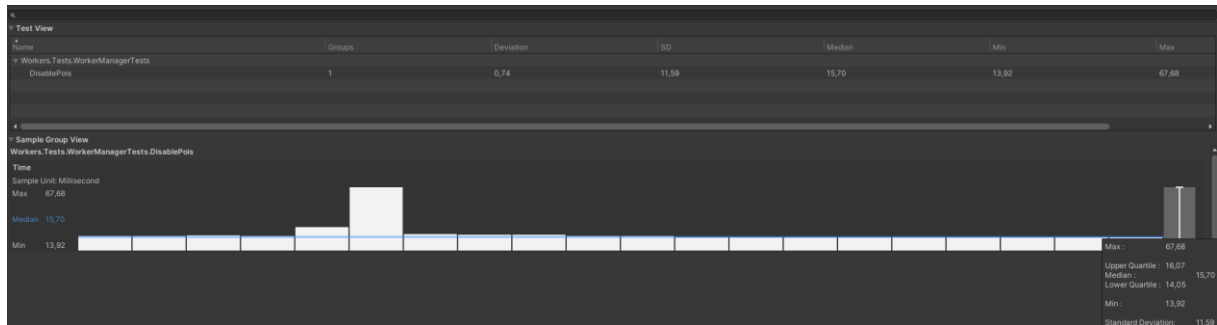


Figure 17: Disable 1 POI.

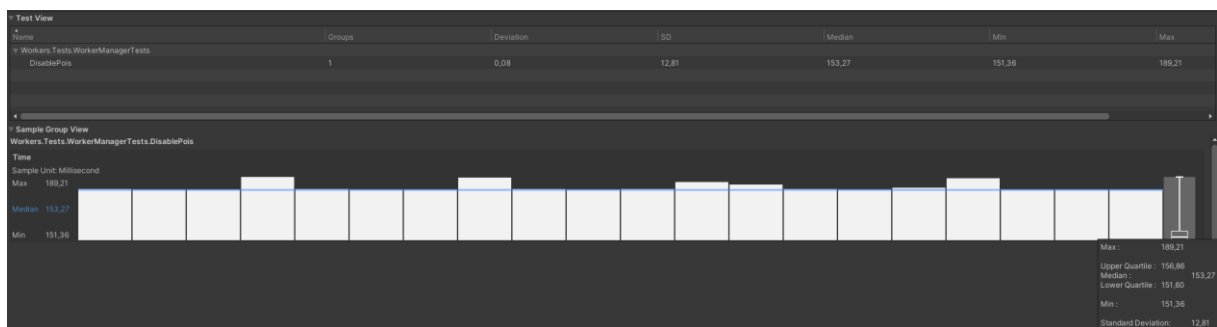


Figure 18: Disable 11 POI's.

4.3.1.2 Set Content (With condition)

The following graph (Figure 19 and 20) shows the operation of **ExecuteWorker("SetContent")** it's job is to set the modal content based upon which POI, if that pois content should be set. Figure 19 shows the content being set when 1 poi is present, and Figure 20 shows the content being set when 11 pois are present.

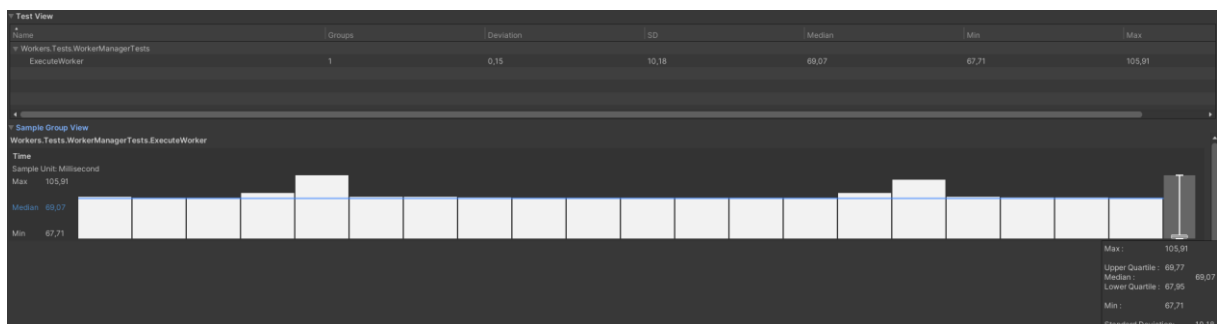


Figure 19: Setting content with 1 POI.

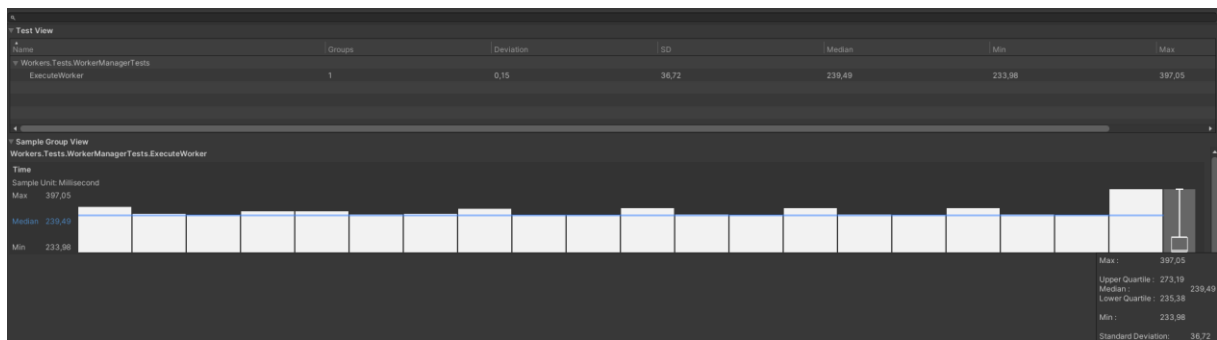


Figure 20: Setting content with 11 POI's.

4.3.2 The Sequence System

4.3.2.1 Move Next Step

The following graph (Figure 21) shows the operation of **MoveNextStep()** it's job is to move towards the next step of the sequence, and call the unity event **OnStepEntered**, **OnStepExited**, **OnAnyStepEntered** and **OnAnyStepExited**.

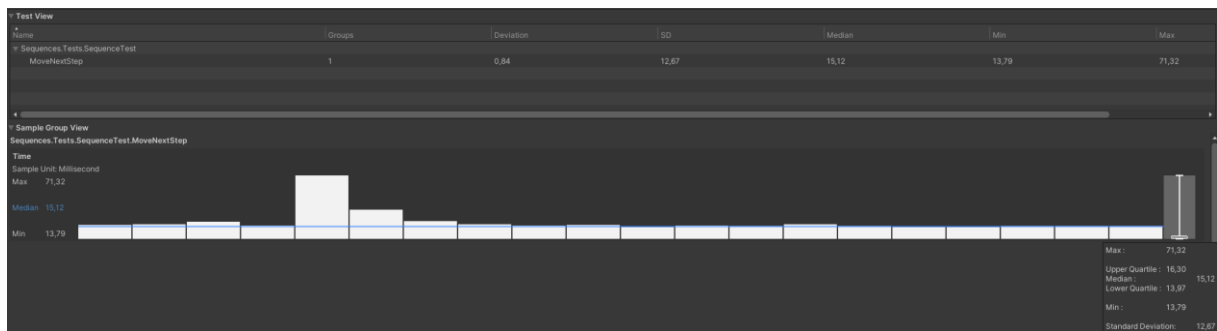


Figure 21: Moving to a next step.

4.3.2.2 Sequence Await Condition Int

The following graph (Figure 22) shows the operation of **CheckValue()** it's job is to check the value to the set value and see if they have an equal sign (More then, less then, equals, or is near.).

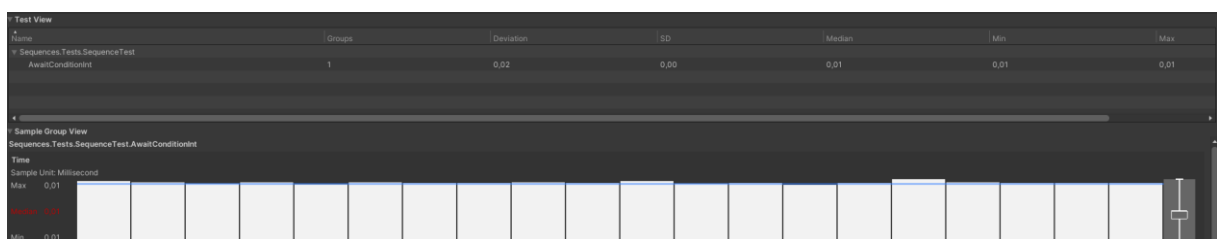


Figure 22: CheckValue Method.

4.3.3 The Transmission System

The following graph (Figure 23) shows the operation of **InvokeTransmission()** it's job is to communicate with its responsible **Transmission Connector** to perform a method inside a just instantiated prefab, or to perform a method affecting the scene.

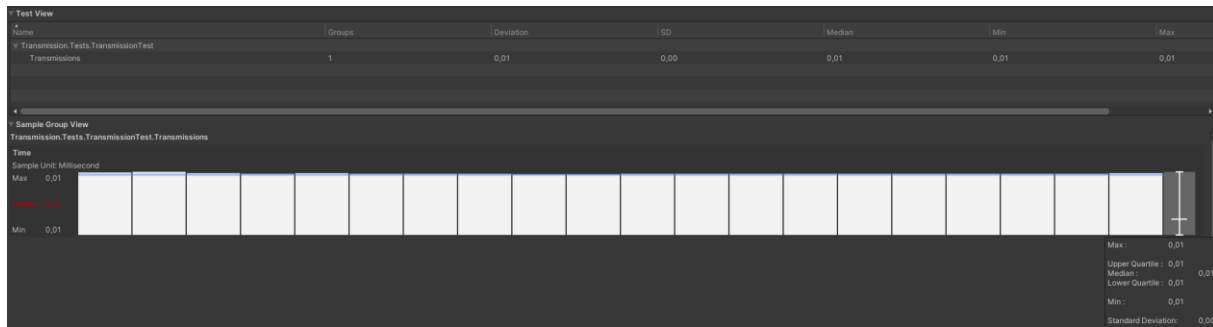


Figure 23: InvokeTransmission Method.

4.3.4 Technical Test Retrospective

The **Sequence System** and the **Transmission System** performed surprisingly well. Even under the load of using reflection, they took no time to execute their methods. The **Worker System** however has a lot left to be desired.

In the graphs shown, the **Sequence System** and **Transmission System** took below 1 millisecond to perform their operations with the given testing parameters. The **Worker System** under the same pressure took over a 100 milliseconds, for reference that is more than **6 frames** in a 60 FPS game.

Once the **Worker System** reaches the end of its execution, under load, it becomes ineffective and causes bottlenecks in framerate. A thorough review will be needed to see where improvements in the **Worker System** can be made.

4.4 User Tests

To User Test the app, I first need to develop a mockup test to do so. This mockup test was tested before testing with the users. Appropriate changes to the mockup will be done based on the feedback.

4.4.1 The mockup

The mockup test was performed with my internship coach. It consisted of performing the following tasks: Start the app (Figure 24); Click on an Eye (Figure 25); Briefly read the description (Figure 26); Change the values of the device in the modal (Figure 27); Close the modal; Open the bill (Figure 28); Check the total costs of the bill (Figure 28); Close the bill; Open the windmill modal (Figure 26); Change the windmill count (Figure 27); Open the bill (Figure 28); Check the total of the bill again; Print the bill to PDF (Figure 29); Close the PDF view; Save the progress (Figure 30); Close the app; Open the app; Load the new data (Figure 31).

The questions I asked about the experience were as follows: How did the info box feel?; What feedback did you observe when hovering over an eye?; How did it feel to navigate the app?; How did it feel to change the device count?; What do you think about being able to load and save data?; How clear is the PDF presentation in the app?; How clear is the PDF presentation when printed?



Figure 24: Main Screen.



Figure 25: Click on Eye.

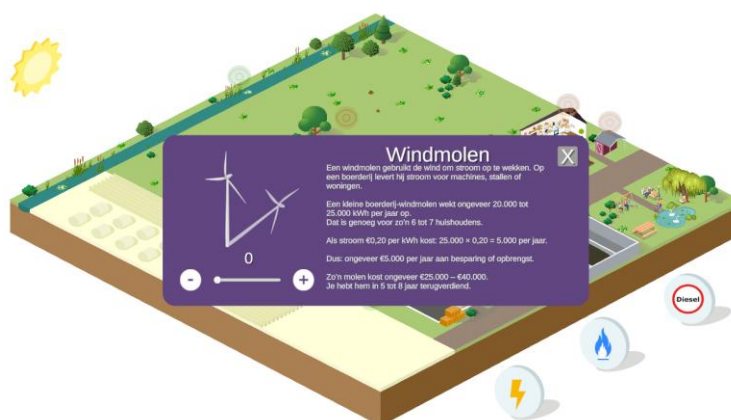


Figure 26: Modal Opened.



Figure 27: Device Count Changed.

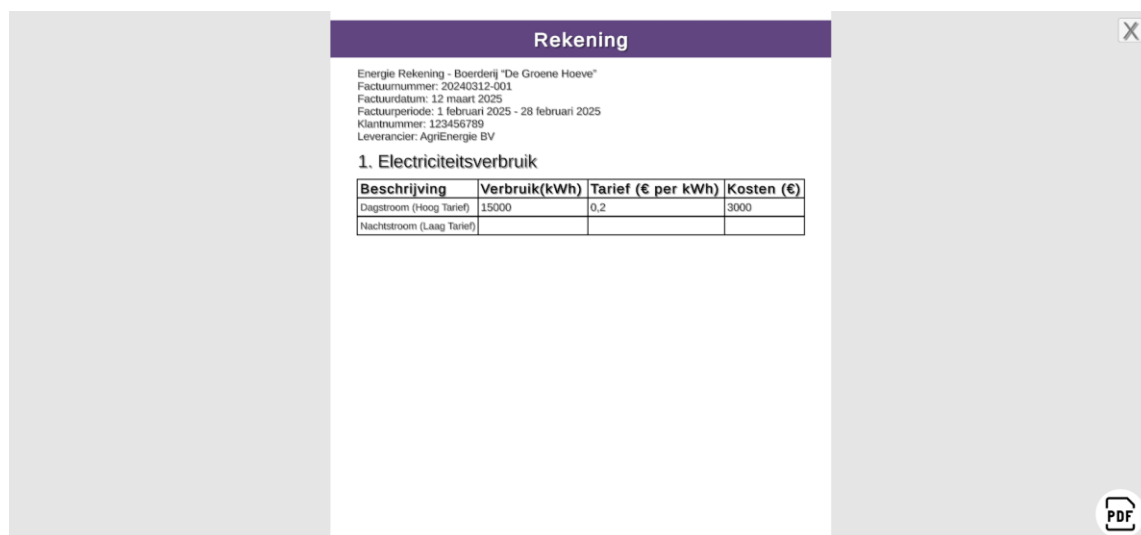


Figure 28: PDF view in app.

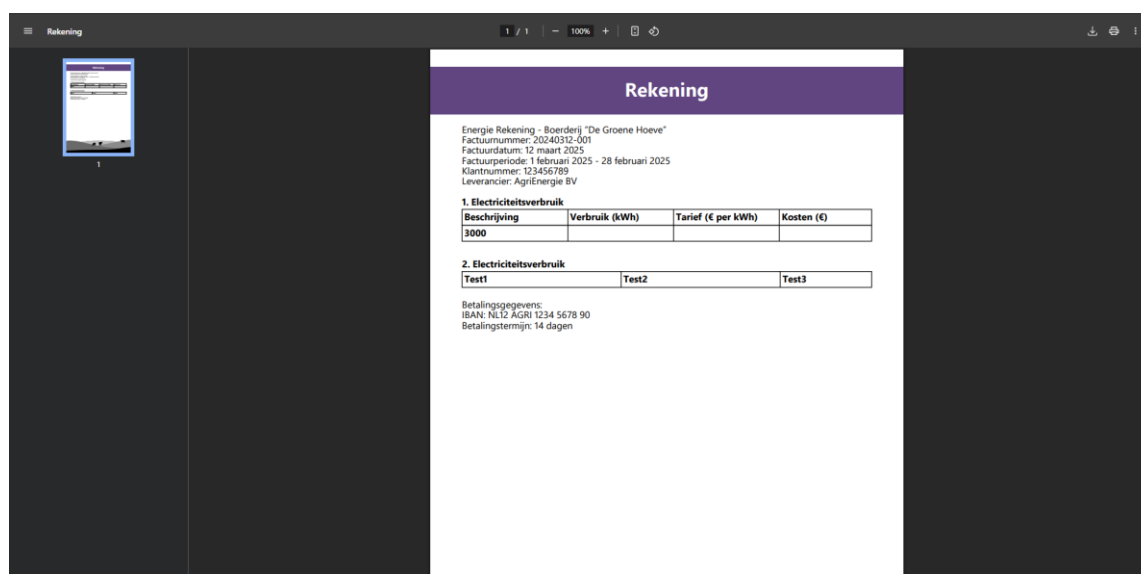


Figure 29: PDF view.

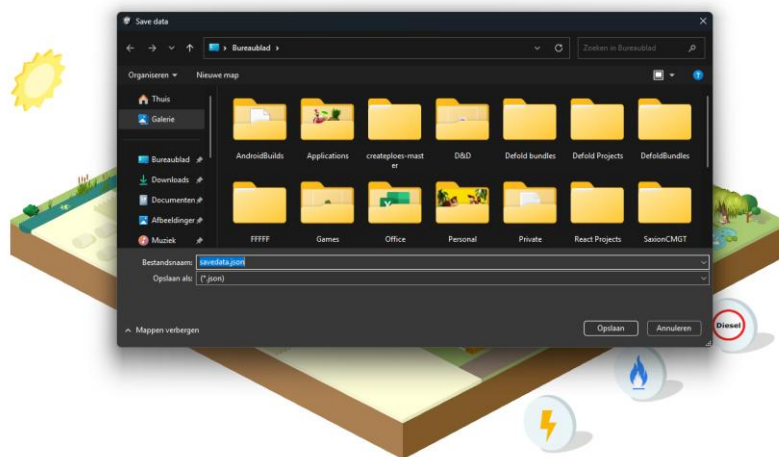


Figure 30: Saving Progress.

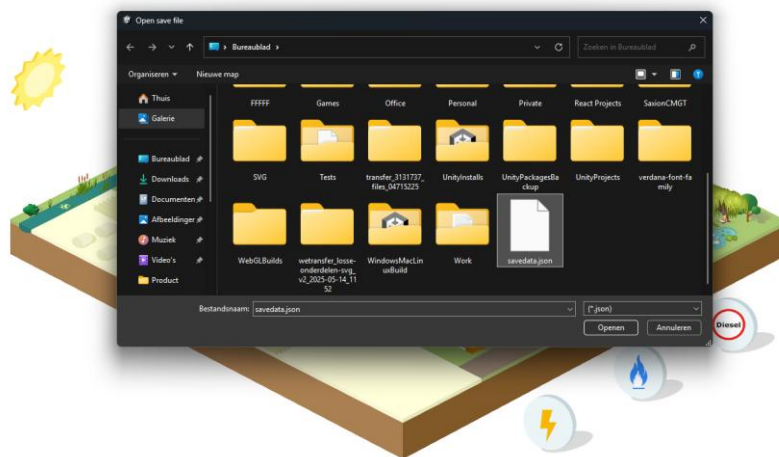


Figure 31: Loading Progress.

4.4.1.1 The mockup test

How did the info box feel?: It feels good, but in my opinion, the font size was a little too small, so it would be desirable if it was a little bigger; **What feedback did you observe when hovering over an eye?:** What you see is for example, when you grab the windmills, that they then appear in the screen. They proceed to do this in a flickering motion, to indicate these are the targets; **How did it feel to navigate the app?:** It feels quite natural, what I do want to pass on, is that at the bottom right of the screen, there is a button for the invoice. I know that that is the button for the invoice, if I did not know that, I would not have found it myself and that also goes for the menu at the top right, the device legend. There is an arrow there, if the text 'overview' can be added that will also help; **How did it feel to change the device count?:** That felt very natural, no further comment; **What do you think about being able to load and save data?:** Look, I couldn't test it, but I have seen that functionality through you and I find that very nice. You can choose where you want to save it, you can choose what name the file gets, it's like opening or exporting a word document, it feel really nice; **How clear is the PDF presentation in the app?:** It is clear. I have only seen one component so I will

have to wait and see how it will look with all of the components. But how it is now, it's very clear; **How clear is the PDF presentation when printed?:** Unfortunately I could not test it, however I have seen it through you, the presentation is comparable with the previous.

The mockup test proceeded with problems. First the save and load system was not working. The same goes for the PDF print system, while it was expected that the PDF print system was not going to work, the breaking of the save and load system is unprecedented. This bug has been since fixed for the oncoming user tests, it was a problem with transforming the app contents into a .zip file when there was no compression enabled in Unity.

Furthermore, the feedback was useful, the mockup tester sounded satisfied. Since then the icon in the bottom right has changed to better reflect it being an invoice button. The suggestion was also given to add a few questions for the internal user test, these are as follows: Was it always clear what you could click on?; How intuitive did you find the buttons and icons?; Were there situations where you waited for feedback, but didn't see anything?; Would you recommend this app to someone with little digital experience?

4.4.2 The User Tests

The following tests were conducted with fellow employees at the company's office. While they are employed there, they have nothing to do with the project as of current. In turn when asked, they knew very little about the project. This prompted me to at least give an explanation of what this application was and who it was for before the actual tests were conducted.

4.4.2.1 User Test 1

How did the info box/modal feel?: I thought it looked nice, clear and well organized. It was nice that there was a sufficient amount of information in the modal; **Was it always clear what you could click on?:** I think so, the only thing was that the close button at the top right must be a bit more clear, especially since we don't know technical the users will be; **How intuitive did you find the buttons and icons?:** They were clear and well-organized, they are recognizable, like the symbols in different applications. I did not know what to really expect with the menu on the top right of the app; **Were there situations where you waited for feedback but didn't see anything?:** Maybe indicate that when you change something in the **Farm Builder** that you get an indication that the invoice has been updated due to your actions. Something like a motivating popup; **How did it feel to navigate the app?:** It felt good, I really like the design, it was clear, calm and not too much fuss in it. The buttons have enough room so that they are not too close together; **How did it feel to change the number of devices?:** It is clear that you should press the + and – button to edit the device count, however to add a layer of clarity, you should specifically explain what the buttons do; **What do you think about being able to store and load your farm environment?:** I think that it's nice that you can share it with colleagues, it seems very easy; **How clear is the presentation of the PDF in the app?:** It's well organized, I can easily see what I want to look for; **How clear is the presentation of the PDF when printed?:** It's also very clear, but the costs are bold, just like the rest of that column, in the other invoice is not the case, so don't make it bold; **Would you recommend this app to someone with little digital experience?:** Yes, but there should be a guide, a short explanation, where they should begin and what different things mean, such as the green/orange eye.

4.4.2.2 User Test 2

How did the info box/modal feel?: It feels good, the eye is easy to find, and the contents are ok, however the text is a little too small. The text is too close together and that makes it hard to read; **Was it always clear what you could click on?:** Loading is not clear, as in the pictogram. The same goes for

the PDF icon, a saving disk does not indicate printing a PDF; **How intuitive did you find the buttons and icons?:** It's clear, most of the pictograms will do what expected, except for the ones just mentioned of course; **Were there situations where you waited for feedback but didn't see anything?:** No I don't think so; **How did it feel to navigate the app?:** It felt well organized, though I did struggle finding the windmills; **How did it feel to change the number of devices?:** Clear; **What do you think about being able to store and load your farm environment?:** It's good, you can save your changes and go back to what you did earlier, and you can then easily experiment with different setups in the **Farm Builder**; **How clear is the presentation of the PDF in the app?:** It's clear, but the text is very close together, there should be more line spacing; **How clear is the presentation of the PDF when printed?:** It's clear, but the same points said in the previous view goes here, as a side note, you could outline the info at the top of the pdf so it would become more readable; **Would you recommend this app to someone with little digital experience?:** As long as you explain how the app works and what you should do, then yeah I would.

4.4.2.3 User Test 3

How did the info box/modal feel?: It felt good, I did need some time to see which eye I should be looking for; **Was it always clear what you could click on?:** For me, it was not clear. The eyes are unclear, as there are more of the same eyes. I took some time for me to find the windmills, saving and the bill was not to clear for me; **How intuitive did you find the buttons and icons?:** It was fine; **Were there situations where you waited for feedback but didn't see anything?:** No I have not waited for feedback in the app, but I just sometimes did not know where to click so I asked for your help; **How did it feel to navigate the app?:** It felt good, no further comment; **How did it feel to change the number of devices?:** A bit strange, you don't really know what it specifically does but other than that, good; **What do you think about being able to store and load your farm environment?:** Good; **How clear is the presentation of the PDF in the app?:** Clear; **How clear is the presentation of the PDF when printed?:** Clear; **Would you recommend this app to someone with little digital experience?:** I would, but it would need to be a bit more clear of what you should do.

4.4.3 User Tests retrospective

The user tests went fine, there was one technical issue related to timelines creating lag. It fully worked in its functionality with no problems. The key points that need attention are as follows: the pictograms need to have their functionality abundantly clear. If it's not, or even slightly off from what the functionality intends, it causes confusion; Navigation in the app was overall good, but the actual 'what do I have to do' part is not clear. Such as going to the windmills and changing their amount to see that reflected in the invoice. A tutorial or an explanation might be needed; Right now, there were already some elements in place on the farm. This is a problem though, if the user wants to look for the windmills on the farm, and none are there, how does the user find it in one clear overview?

Later when looking into the technical issue, the lag originated from the experimental SVG library. This was because every time an alpha change was made to the SVG, it was completely redrawn from scratch. This was fixed by handling its opacity values through a canvas group rather than directly editing its color.

These key points will need to be addressed into the app. But for now, that is out of scope for this project, except for the performance point, but that has been fixed. Many of the points noted are related to UX design and should be better addressed by a UX expert.

5. Reflection, Conclusion & Future Work

5.1 Conclusion

This document describes the process of creating the **Farm Builder** application. A prototype was made where you can see the impacts of energy consumption and production on a farm and inside of a PDF bill in Windows.

A performance test was conducted on the most intensive methods inside of the **Farm Builder**, there is room for improvement, specifically related to the reflection aspect of the systems I created with regards to the **Worker** system.

User tests were conducted internally within the company, which showed that there is now a need for a UX designer to improve the overall user experience on the application.

5.1.1 Main Research Question

How can software architecture be used to design and implement the Farm Builder system to guarantee a modular codebase that is dynamic and easily adapts to new features being introduced allowing for users to save/load their progress and view the financial side of the farm in PDF form, and help future developers maintain and expand upon the codebase in an easy manner?

It can be done by keeping small mechanics inside of the editor itself and in the scene to prevent bloat inside of the codebase. The systems I have created have helped me as a developer to accelerate the development process of the **Farm Builder** and help me keep easy track of certain features inside of the app. These systems were used to create a modular approach for saving, loading and bringing the PDF bill into fruition. Everything in the app can be changed by changing values in the editor, or by simple drag and drop.

5.1.2 How to make a modular codebase?

To make a modular codebase, inside of Unity goes a bit further than reusability. Unity provides an editor, so that should be used to the developers advantage to make the codebase even more modular. In this project I have created many different systems that have their entire implementation based upon setup in the editor. This does provide quite the abstraction and at times performance heavy operations (like the Worker System) but it does provide a way to quickly iterate over different ideas and new mechanics.

5.1.3 How to save and load inside of the application in a dynamic and adaptable way?

With the help of the 1st sub question answered, it is possible to create a save and load system by introducing the ability to save and load data with drag and drop functionality. Allowing for quick adaptation to new data being present in the game that needs to be persistent. It is also possible to push this feature even further by introducing a native file explorer library, namely **StandaloneFileBrowser** (Gökçe, G. (2017)) to handle the file saving and file loading of the save file.

5.1.4 How to export the bill into PDF form, both for windows and web?

Exporting the bill in PDF has been a difficult process, but it is possible for windows and web by implementing different methods such as using different kinds of libraries, namely PDFsharp-MigraDoc

(Empira Software GmbH, n.d.) and PDFKit (PDFKit, n.d.) to take that process over, or switching to an entirely different programming language to write that implementation yourself (WebGL).

5.2 Future work

As of current, I am already working on optimizing and refactoring different systems, some towards a more ease of use, like the Transmission system, or enhancing the performance of certain methods, like for the Worker system. PDF in web should be fully implemented. A UX designer would be invited to look at the application and improve it on user experience.

5.3 Reflection

My strongest point by far is being curious. Curiosity is what drives me to improve, and to look back at what I have done, and make improvements upon my components. Willpower plays a part in this to, otherwise I would not be able to see the solution through to the end.

An example of this trait, is with the transmission system. My graduation teacher suggested to me that it might be a good idea to translate that system into a scriptable object based approach. This is due to the fact that the transmission system worked with a string based approach, which is really unsafe and very easily prone to mistakes. To resolve this, I went to research it, if it was possible. And it very much was, but with a very large layer of added complexity, which I wanted to avoid. I from then on started brainstorming on how to improve that system, and I finally did think of a solution, which is still prone to mistakes, but much safer and clear on how to approach it.

This does have its negative side effects though, such as going too deep into a solution that might not have a beneficial outcome at all, which just means I wasted my time. This project however, I did manage to keep that under control, if it took a long time, it really needed to be worth it. Otherwise I would need to find a different solution or keep this one.

I could have improved upon the planning and testing of the product, maybe I should've spoken up more about the need for the content of the application, this is because it would have maybe given me room to actually test with the target audience, I already spoke about it quite a lot but, more pushing at an earlier time would maybe be beneficial next time.

The performance testing was done in an excellent manner this time, I would keep testing very barebones, but with reflection at play it was very important to track its performance impact, especially since at the time it was being used at runtime.

Resources

- Aeres Tech Productions. (n.d.). *Over ons* [About us page]. <https://www.aerestechproductions.nl/over-ons>
- Amazon Web Services. (n.d.). *What is code quality?* <https://aws.amazon.com/what-is/code-quality/>
- Empira Software GmbH. (n.d.). *Overview of MigraDoc (Library)*. PDFsharp & MigraDoc Documentation. <https://docs.pdfsharp.net/MigraDoc/Overview/About.html>
- Garlan, D. (2008). *Software architecture* [PDF]. Carnegie Mellon University KiltHub. https://kilthub.cmu.edu/articles/journal_contribution/Software_Architecture/6609593/files/12101711.pdf
- Google. (n.d.). *C# at Google: C# style guide* [Style guide]. <https://google.github.io/styleguide/csharp-style.html>
- Gökçe, G. (2017). *UnityStandaloneFileBrowser* [Computer software]. GitHub. <https://github.com/gkngkc/UnityStandaloneFileBrowser>
- Microsoft. (2025, February 10). *C# coding conventions* [Documentation]. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>
- Pham, D. V. (2020). *Game architectures in Unity projects* (Bachelor's thesis, Turku University of Applied Sciences). https://www.theseus.fi/bitstream/handle/10024/354965/Duy_Vu.pdf
- PDFKit. (n.d.). *PDFKit: A JavaScript PDF generation library for Node and the browser*. <https://pdfkit.org/>
- Unity Technologies. (n.d.-a). *Unity performance testing extension 3.1 manual*. Unity. <https://docs.unity3d.com/Packages/com.unity.test-framework.performance@3.1/manual/index.html>
- Unity Technologies. (n.d.-b). *Measure.Method | Performance Testing Package manual* [Documentation]. <https://docs.unity3d.com/Packages/com.unity.test-framework.performance@3.1/manual/measure-method.html>
- Venters, C. C., Capilla, R., Betz, S., Penzenstadler, B., Crick, T., Crouch, S., Nakagawa, E. Y., Becker, C., & Carrillo, C. (2018). *Software Sustainability: Research and Practice from a Software Architecture Viewpoint*. *Journal of Systems and Software*, 138, 174–188. <https://doi.org/10.1016/j.jss.2017.12.026>
- .NET Foundation. (n.d.). *Coding style* [Computer software documentation]. In *dotnet/runtime* (docs/coding-guidelines). GitHub. <https://github.com/dotnet/runtime/blob/HEAD/docs/coding-guidelines/coding-style.md>